

MapReduce 原理及其主要实现平台分析^{*}

亢丽芸 王效岳 白如江

(山东理工大学科技信息研究所 淄博 255049)

【摘要】针对海量数据处理在处理速度、存储空间、容错性、访问时间等方面存在的问题,对 Google MapReduce 编程模型的原理、执行流程等进行分析研究,介绍 4 种主要的 MapReduce 实现平台 Hadoop、Phoenix、Disco、Mars,从编程语言、构建平台、功能特点和应用领域 4 个方面对 4 种平台进行比较分析,以期对 MapReduce 编程模型原理及其应用平台有一个较全面的认识。

【关键词】MapReduce 实现平台 Hadoop Phoenix Disco Mars

【分类号】G250 TP391

Analysis of MapReduce Principle and Its Main Implementation Platforms

Kang Liyun Wang Xiaoyue Bai Rujiang

(Institute of Scientific & Technical Information, Shandong University of Technology, Zibo 255049, China)

【Abstract】Due to the problems in processing speed, storage space, fault tolerance, access time and others of massive data processing, this paper analyzes principle, implementation process of Google MapReduce programming model, introduces four main MapReduce implementation platforms, including Hadoop, Phoenix, Disco and Mars. Then, it separately compares them in four aspects as the programming language, building platform, functions and features, applications to have a comprehensive understanding of MapReduce programming model and it's application platforms.

【Keywords】MapReduce Implementation platform Hadoop Phoenix Disco Mars

1 引言

随着 Internet 网络资源的迅速膨胀,因特网容纳了海量的各种类型的数据和信息。海量数据的处理对服务器 CPU、IO 的吞吐都是严峻的考验,不论是处理速度、存储空间、容错性,还是在访问速度等方面,传统的技术架构和仅靠单台计算机基于串行的方式越来越不适应当前海量数据处理的要求。国内外学者提出很多海量数据处理方法,以改善海量数据处理存在的诸多问题。目前已有的海量数据处理方法在概念上较容易理解,然而由于数据量巨大,要在可接受的时间内完成相应的处理,只有将这些计算进行并行化处理,通过提取出处理过程中存在的可并行工作的分量,用分布式模型来实现这些并行分量的并行执行过程。随着技术的发展,单机的性能有了突飞猛进的发展变化,尤其是内存和处理器等硬件技术,但是硬件技术的发展在理论上总是有限度的,如果说硬件的发展在纵向上提高了系统的性能,那么并行技术的发展就是从横向上拓展了处理的方式。

MapReduce^[1]是由谷歌推出的一个编程模型,也是一个能处理和生成超大数据集的算法模型,该架构能够在大量普通配置的计算机上实现并行化处理。

收稿日期:2011-09-21

收修改稿日期:2012-01-23

^{*} 本文系国家社会科学基金一般项目“海量网络学术文献自动分类研究”(项目编号:10BTQ047)和山东省自然科学基金项目“大规模学术文献并行处理与语义分类研究”(项目编号:ZR2011GL025)的研究成果之一。

2 MapReduce 编程模型研究

2.1 MapReduce 编程模型原理

MapReduce 编程模型结合用户实现的 Map 和 Reduce 函数,可完成大规模的并行化计算。MapReduce 编程模型的原理^[1]是:用户自定义的 Map 函数处理一个输入的基于 key/value pair 的集合,输出中间基于 key/value pair 的集合,MapReduce 库把中间所有具有相同 key 值的 value 值集合在一起后传递给 Reduce 函数,用户自定义的 Reduce 函数合并所有具有相同 key 值的 value 值,形成一个较小 value 值的集合。一般地,一个典型的 MapReduce 程序的执行流程如图 1 所示^[2]:

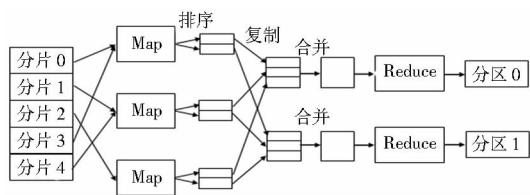


图 1 MapReduce 执行流程

MapReduce 执行过程主要包括:

- (1) 将输入的海量数据切片分给不同的机器处理;
- (2) 执行 Map 任务的 Worker 将输入数据解析成 key/value pair,用户定义的 Map 函数把输入的 key/value pair 转成中间形式的 key/value pair;
- (3) 按照 key 值对中间形式的 key/value 进行排序、聚合;
- (4) 把不同的 key 值和相应的 value 集分配给不同的机器,完成 Reduce 运算;
- (5) 输出 Reduce 结果。

任务成功完成后,MapReduce 的输出存放在 R 个输出文件中,一般情况下,这 R 个输出文件不需要合并成一个文件,而是作为另外一个 MapReduce 的输入,或者在另一个可处理多个分割文件的分布式应用中使用。

2.2 MapReduce 的类型与格式

MapReduce 的数据模型较简单,它的 Map 和 Reduce 函数使用 key/value pair 进行输入和输出,Map 和 Reduce 函数遵循的形式如表 1 所示^[1]。

MapReduce 库支持多种不同格式的输入数据类型,如文本模式的输入数据,每一行被视为一个key/

表 1 Map/Reduce 函数数据模型

函数	输入	输出
Map	(k1 ,v1)	list(k2,v2)
Reduce	(k2 ,list(v2))	list(k3,v3)

value pair,key 是文件的偏移量,value 是该行的文本内容^[1]。MapReduce 的预定义输入类型能够满足大多数的输入要求,使用者还可通过提供一个简单的 Reader 接口,实现一个新的输入类型。MapReduce 还提供了预定义的输出类型,通过这些预定义类型能够产生不同格式的输出数据,用户可采用类似添加新输入数据类型的方式增加新输出类型。

3 MapReduce 主要实现平台

受 Google MapReduce 启发,许多研究者在不同的实验平台上实现了 MapReduce,并取得了一些研究成果,其中较具代表性的研究成果有 Apache 的 Hadoop,斯坦福大学的 Phoenix,Nokia 研究中心的 Disco 和香港科技大学的 Mars。

3.1 Hadoop

(1) 平台介绍

Apache 软件基金会对 Hadoop (<http://hadoop.apache.org/>) 的设计思想,来源于 Google 的 GFS (Google File System)^[3] 和 MapReduce^[1]。Hadoop 是一个开源软件框架,通过在集群计算机中使用简单的编程模型,可编写和运行分布式应用程序处理大规模数据。Hadoop 包括三个子项目: Hadoop Common、Hadoop Distributed File System (HDFS) 和 Hadoop MapReduce,其中 Hadoop Common 是一套支持 Hadoop 子项目的工具,由文件系统、远程过程调用 (Remote Procedure Calls , RPC) 和序列化库构成; HDFS 是一个分布式文件系统,提供高吞吐量的应用程序数据访问; Hadoop MapReduce 是一个在计算机集群上分布式处理海量数据集的软件框架。

(2) 工作流程

Hadoop 运行 MapReduce 作业的整个过程如图 2 所示^[2]。

在最上层,有 4 个独立的实体,客户端、jobtracker、tasktracker 和分布式文件系统。客户端提交 MapReduce 作业; jobtracker 协调作业的运行, jobtracker 是一个 Java 应用程序,它的主类是 JobTracker; tasktracker 运行作业划分后的任务, tasktracker 也是一个 Java 应用程

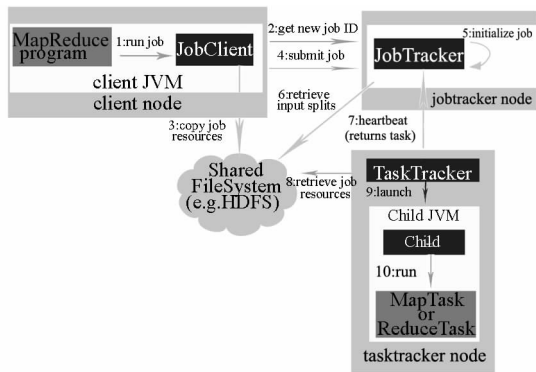


图2 Hadoop 运行 MapReduce 作业工作原理^[2]

序,它的主类是 TaskTracker;分布式文件系统(一般为 HDFS)用来在其他实体之间共享作业文件。Hadoop 运行 MapReduce 作业的步骤主要包括:提交作业;初始化作业;分配任务;执行任务;更新进度和状态;完成作业^[2]。

Hadoop MapReduce 框架包括一个 Jobtracker 和一定数量的 Tasktracker^[4],Jobtracker 通常运行在和名称节点相同的主机上。用户将 MapReduce 作业发送给 Jobtracker 所在集群的其他机器上分割工作,集群中其他的空闲机器,每个机器运行一个 Tasktracker,Tasktracker 与 Jobtracker 通信,在可能的情况下,Jobtracker 给它们分配 Map 或 Reduce 任务。

3.2 Phoenix

(1)平台介绍

Phoenix(<http://mapreduce.stanford.edu/>)作为斯坦福大学 EE382a 课程^[5]的一类项目,由斯坦福大学计算机系统实验室开发。Phoenix^[6]是为达到共享内存系统的目的,对 Google MapReduce 的一种实现方式。Phoenix 对 MapReduce 的实现原则和最初由 Google 实现的 MapReduce 基本相同。不同的是,它在集群中以实现共享内存系统为目的,共享内存能最小化由任务派生和数据间的通信所造成的间接成本。Phoenix 可编程多核芯片或共享内存多核处理器(SMPs 和 ccNUMAs),用于数据密集型任务处理。

(2)工作流程

Phoenix 系统包括一个简单的应用程序编程接口(Application Programming Interface, API)和一个高效运行时系统,该 API 对应用程序的程序员是可见的,高效的运行时系统能够自动管理线程的创建、动态任务调度、数据分区、跨处理器节点的容错。Phoenix 使用线

程创建并行化的 Map 和 Reduce 任务,给可用的处理器动态调度任务,为了实现负载均衡和最大化吞吐量,通过调整任务粒度和并行任务的分配来进行局部性管理。

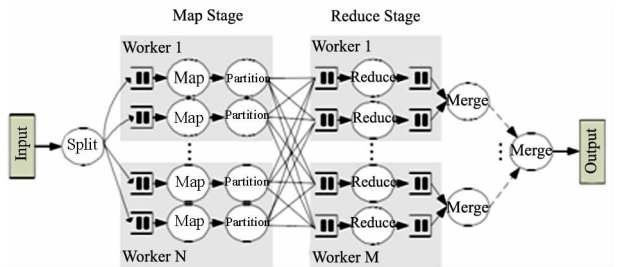


图3 Phoenix 运行时系统的基本数据流^[6]

图3显示了 Phoenix 运行时系统工作的基本流程^[6],运行时由调度器控制,调度器由用户代码进行初始化。调度器创建和管理所有运行 Map 和 Reduce 任务线程,还管理用于任务通信的缓冲区。程序员通过 scheduler_args_t 结构体来初始化调度器需要的所有数据和函数指针,初始化完成后,调度器判定用于计算的内核数量,对于每一个内核,它派生一个线程,动态分配一定数量的 Map 和 Reduce 任务^[6]。

3.3 Disco

(1)平台介绍

Disco(<http://discoproject.org/>)是由 Nokia 研究中心开启的,基于 MapReduce 的分布式数据处理框架,核心部分由并行性很高的 Erlang 语言开发,外部编程接口为 Python 语言。Disco 是一个开放源码的大规模数据分析平台,支持大数据集的并行计算,能运行在不可靠的集群计算机上。Disco 可部署在集群和多核计算机上,还可部署在 Amazon EC2 上。

(2)工作流程

Disco 基于主/从(Master/Slave)架构,总体设计架构如图4所示^[7]:

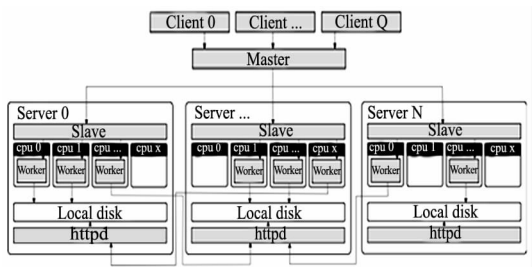


图4 Disco 总体设计架构^[7]

Disco 工作流程主要包括5个部分:

- ①Disco 用户使用 Python 脚本开始 Disco 作业;
- ②作业请求通过 HTTP 发送到主机;
- ③主机是一个 Erlang 进程,通过 HTTP 接收作业请求;
- ④主机通过 SSH 启动每个节点处的从机;
- ⑤从机在 Worker 进程中运行 Disco 任务。

客户端进程是一些 Python 程序,使用函数 disco.job() 向 Master 提交作业;主机 Master 接收作业,并将它们添加到作业队列中,以便进行调度;当集群中的节点可用时,Master 启动集群中的 Slave 机器,Slave 机器在各自的节点处启动和监控所有的进程;Workers 用于执行提交作业中的具体任务,Workers 的输出位置被发送到 Master。Disco 在每个从机服务器节点上运行一个 HTTP 服务器,当 Worker 和输入运行在不同节点时,便于数据的远程访问。用户可以限制每个节点上并行运行的 Worker 数量,根据集群中的可用 CPU 和磁盘资源,指定尽可能多的任务。

3.4 Mars

(1) 平台介绍

一些研究者正在将 MapReduce 架构扩展到图形处理器(Graphic Processing Units, GPUs)上实现,香港科技大学的 He 等^[8],与微软和新浪合作,在单 GPU 上开发了 Mars(<http://www.cse.ust.hk/gpuqp/Mars.html>)。Mars 是基于图形处理器(GPUs)对 Google MapReduce 的一个实现,目前已经包含字符串匹配、矩阵乘法、倒排索引、字词统计、网页访问排名、网页访问计数、相似性评估和 K 均值等 8 项应用,能够在 32 位与 64 位的 Linux 平台下运行^[9]。

(2) 工作流程

Mars 和基于 GPU 的 MapReduce 框架相似,也包括 Map 和 Reduce 两个阶段,GPU 上 Mars 的基本工作流程如图 5 所示^[8]:

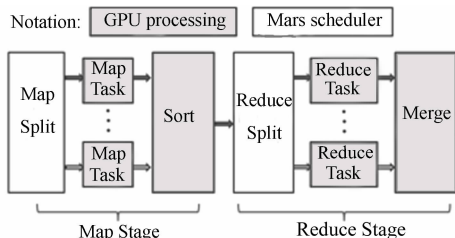


图 5 GPU 上 Mars 的基本工作流程^[8]

在开始每个阶段之前,Mars 初始化线程配置,包括 GPU 上线程组的数量和每个线程组中线程的数量。

Mars 在 GPU 内使用大量的线程,在运行时系统的时候,任务均匀分配给线程,每个线程负责一个 Map 或 Reduce 任务,以小数量的 key/value pairs 作为输入,并通过一种无锁的方案来管理 MapReduce 框架中的并发写入。

Mars 的工作流程主要有 7 个操作步骤:

- ①在主存储器中输入 key/value 对,并将它们存储到数组;
- ②初始化运行时的配置参数;
- ③复制主存储器中的输入数组到 GPU 设备内存;
- ④启动 GPU 上的 Map 阶段,并将中间的 key/value 对存储到数组;
- ⑤如果 noSort 选择 F,即需要排序阶段,则对中间结果进行排序;
- ⑥如果 noReduce 是 F,即需要 Reduce 阶段,则启动 GPU 上的 Reduce 阶段,并输出最终结果,否则中间结果就是最终结果;
- ⑦复制 GPU 设备存储器中的结果到主存储器。

在上述步骤中,①②③和⑦的操作由调度器来完成,调度器负责准备数据输入,在 GPU 上调用 Map 和 Reduce 阶段,并将结果返回给用户。

4 主要实现平台的比较分析

4.1 编程语言

(1) Hadoop

Hadoop 是采用 Java 开发的,所以能很好地支持 Java 语言编写的 MapReduce 作业,但在实际应用中,有时候由于要用到非 Java 的第三方库或者其他原因,需采用 C/C++ 或其他语言编写 MapReduce 作业,这时候可能要用到 Hadoop 提供的一些工具。若用 C/C++ 编写 MapReduce 作业,可使用 Hadoop Streaming^[10] 或 Hadoop Pipes^[2,11] 工具;若用 Python 编写 MapReduce 作业,可以使用 Hadoop Streaming 或 Pydoop^[12,13] 工具;若使用其他语言,如 Shell 脚本、PHP、Ruby 等,可使用 Hadoop Streaming。

Java 是 Hadoop 支持的最好最全面的语言,而且提供了很多工具方便程序员开发。Hadoop 流(Hadoop Streaming)使用 Unix 标准流作为 Hadoop 和程序之间的接口,其最大的优点是支持多种编程语言,只要编写的 MapReduce 程序能够读取标准输入,并写到标准输出,但效率较低,Reduce 任务需等到 Map 阶段完成后才能

启动。Hadoop 管道 (Hadoop Pipes) 是 Hadoop MapReduce 的 C++ 接口的代称, 与流不同, 流使用标准输入和输出让 Map 和 Reduce 节点之间相互交流, 管道使用 sockets 作为 tasktracker 与 C++ 编写的 Map 或者 Reduce 函数的进程之间的通道。Pydoop 是专门为 Python 程序员编写 MapReduce 作业设计的, 底层使用了 Hadoop Streaming 接口和 libhdfs 库。

(2) Phoenix

目前,Phoenix 能够提供 C 和 C++ 的应用程序编程接口,类似的 API 也可以被定义成 Java 或者 C# 这样的语言^[6]。Phoenix API 包括两部分函数集:第一个函数集由 Phoenix 提供,可用于应用程序代码中来初始化系统和发出输出对;第二部分包括程序员自定义函数。

(3) Disco

Disco 平台的核心部分由并行性能很高的 Erlang 语言开发,其外部编程接口为易于编程的 Python 语言。Disco 代码风格指南^[14]包含了 Disco 代码库的编码约定:对于 Erlang 而言,除另有规定外,一般采用 Erlang 编程规则^[15];对于 Python,除了另有规定外,采用 PEP 8 准则^[16]。

(4) Mars

目前 Mars 提供了 C 和 C++ 的应用程序编程接口 API,和已有的 MapReduce 框架相似,Mars 也有两种 APIs 集:一种是系统提供的 APIs,用户可以通过调用库来使用;另一种是用户提供的 APIs,该部分由用户实现^[8]。Mars 当前版本可运行在 32 位和 64 位的 Linux 系统上,支持最新的 CUDA SDK 2.3。

上述 4 种平台的 API 支持的编程语言比较如表 2 所示:

表2 MapReduce 不同实现平台编程语言

平台	Hadoop	Phoenix	Disco	Mars
编程语言	Java、C、C++、Python、 Shell、PHP、Ruby	C、C++	Erlang、 Python	C、C++

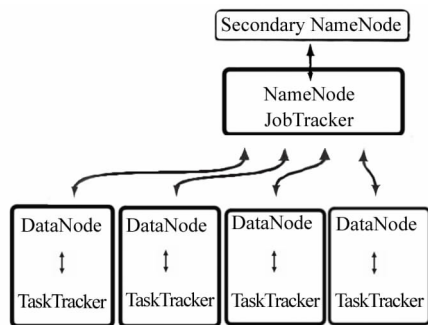
4.2 构建平台

(1) Hadoop

Hadoop 平台搭建完成后,在一个全配置的集群中,运行 Hadoop 意味着在集群的不同机器上运行一组守护进程(daemons),这些进程包括:名称节点(NameNode):数据节点(DataNode):次名称节点(Secondary

NameNode); 作业跟踪节点 (JobTracker); 任务跟踪节点 (TaskTracker)。

从 HDFS 分布式存储的角度来说,集群中的节点由一个 NameNode 和若干个 DataNode 组成,另有一个 Secondary NameNode 作为 NameNode 的备份;从 MapReduce 分布式计算的角度来说,集群中的节点由一个 JobTracker 和若干个 TaskTracker 组成。典型 Hadoop 集群的拓扑结构如图 6 所示^[17]:

图 6 典型 Hadoop 集群拓扑结构^[17]

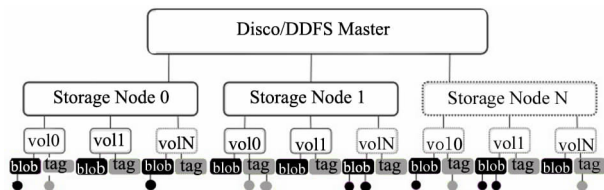
Hadoop 的分布式存储和分布式计算都采用了主/从结构,NameNode 和 JobTracker 的守护进程运行在主节点上,DataNode 和 TaskTracker 运行在从节点上,TaskTracker 必须运行在 DataNode 上,便于数据的本地计算,JobTracker 和 NameNode 则无须一定在同一台机器上。

(2) Phoenix

Phoenix 系统包括一个简单的 API 和一个高效的运行时系统,该 API 对应用程序的程序员是可见的,高效的运行时系统能够自动管理线程创建、动态任务调度、数据分区、跨处理器节点的容错。运行时系统的实现是建立在 PThread^[18]之上的,也可方便地移植到其他共享内存线程库上。

(3) Disco

Disco 平台由分布式存储系统 DDFS(Disco Distributed File System) 和 MapReduce 框架组成,DDFS 与计算框架是高度耦合的,DDFS 架构如图 7 所示^[19]:

图 7 DDFS 的架构^[19]

DDFS 嵌入到 Disco 内,有一个 Master 节点和多个存储节点,每个存储节点由一组磁盘或者卷宗组成 (vol0...volN),它们分别挂载在 DDFS_DATA/vol0 ... DDFS_DATA/volN 上。每个卷宗下面有两个文件 tag 和 blob,分别用于存储标记 tag (相当于 key) 和标记对应的值 (即 value)。DDFS 会监控各个节点上的磁盘使用情况,每隔一段时间进行负载均衡。

(4) Mars

Mars 在多核架构模型基础上设计,提供了一个小的 APIs 集,这些和基于 CPU 的 MapReduce 相似。Mars 运行时为 Map 或 Reduce 任务初始化大量的 GPU 线程,并为每个线程自动分配少量的 key/value 对来运行任务。由于数据分析任务涉及到大量的文本处理,Mars 的 APIs 还设计了一个高效的字符串库。

4.3 功能特点

(1) Hadoop

Hadoop 是一个开源框架,通过在大规模集群计算机中使用简单的编程模型,可编写和运行分布式应用程序处理大规模数据,是目前应用广泛的开源并行编程框架。

Hadoop 具有诸多优点:

①良好的扩展性:通过简单增加集群节点,可处理更大规模的数据集;

②可靠性:Hadoop 致力于运行在一般商用硬件上,其架构假设硬件会频繁失效,它可以从容地处理大多数故障;

③高效性:Hadoop 集群的存储和计算能力非常强,适合有超大数据集的应用程序;

④经济性:Hadoop 运行在一般商用机器构成的大型集群上或如亚马逊弹性计算云 (EC2) 等云计算服务器之上;

⑤易用性:Hadoop 方便、简单,用户可快速搭建自己的 Hadoop 集群,并编写出高效的并行代码。

但是 Hadoop 也存在一些不足,由于系统开销等原因,处理小规模数据的速度不一定比串行程序快,而且单机处理数据的性能较低;若计算时产生的中间结果文件非常大,Reduce 过程需要通过远程过程调用来获取中间结果文件,这样会加大网络传输开销;作为一个比较新的项目,Hadoop 在很多方面还需提升,包括稳定性、易用性、可维护性、可测试性等,特别是在 MapReduce 层,还未解决线性扩展问题。

(2) Phoenix

Phoenix 在共享内存系统上实现了 MapReduce,利

用共享内存缓冲区实现通信,从而避免了因数据复制产生的开销,但 Phoenix 也存在不能自动执行迭代计算、没有高效的错误发现机制等不足之处^[20]。

(3) Disco

Disco 是一个开源的大规模数据分析平台,由一个 Master 服务器和一系列 Worker 节点组成,其中 Master 和 Worker 之间采用基于轮询^[21]的通信机制,使用 HTTP 的方式传输数据。Master 服务器时刻跟踪用户应用,负责任务调度与分配,保存与应用相关的输入数据、中间值和最终结果。Worker 节点则负责执行 Map 与 Reduce 任务,但是由于轮询的时间间隔不好确定,若时间间隔设置不当,则会显著降低程序的执行性能^[20]。

(4) Mars

Mars 是基于 GPUs 的 MapReduce 实现,该框架旨在为开发者提供一个基于 GPU 的通用框架以完成准确、高效、简单的实施数据和计算密集型任务。由于 GPU 线程不支持运行时动态调度,所以给每个 GPU 线程分配的任务是固定的,若输入数据划分不均匀,将导致 Map 或 Reduce 阶段的负载不均衡,使得整个系统性能急剧降低。同时由于 GPU 不支持运行时在设备内存中分配空间,需要预先在设备内存中分配好输入数据和输出数据的存放空间,但是 Map 和 Reduce 阶段输出数据大小是未知的,并且当多个 GPU 线程同时向共享输出区域中写数据时,易造成写冲突^[20]。

MapReduce 不同的实现平台和研究成果有着不同的功能和优缺点,主要实现平台功能特点比较如表 3 所示。

4.4 应用领域

(1) Hadoop

Hadoop 可充分利用集群的优势进行高速运算和存储,由于 Hadoop 在处理大规模数据上的优势,基于 Hadoop 的应用非常多,尤其是在互联网领域。Yahoo! 通过集群运行 Hadoop,以支持广告系统和 Web 搜索研究,并将 Hadoop 广泛应用于日志分析、广告计算、科研实验中^[22]; Amazon 的搜索门户 A9.com 基于 Hadoop 完成了商品搜索的索引生成^[23]; 互联网电台和音乐社区网站 Last.fm 使用 Hadoop 集群进行日志分析、A/B 测试评价、Ad Hoc 处理和图表生成等日常作业^[24]; 著名 SNS 网站 Facebook 使用 Hadoop,存储日志数据,支

表3 不同实现平台功能特点比较

平台名称	主要功能	主要优点	主要缺点
Hadoop	开发、运行和处理大规模数据的平台	良好的扩展性、可靠性、高效性和经济性;集群的存储和计算能力非常强,适合有超大数据集的应用程序;应用广泛	对小规模数据处理速度低;若中间结果文件过大,会加大通信开销;MapReduce层的线性扩展能力较差;单机的性能较低
Phoenix	基于共享内存系统实现 MapReduce 模型来执行数据密集型处理任务	利用共享内存缓冲区实现通信,避免数据拷贝产生的开销	不支持自动迭代计算,缺乏高效的错误发现机制
Disco	大规模数据分析平台	使用 HTTP 的方式传输数据;高效的数据局部性保留 IO;可创建和查询数十亿 key/value 的索引; Master 和 Worker 之间采用基于轮询的通信机制	DDFS 目前只有一个 Master 节点,存在单点故障; Master 和 Worker 轮询时间间隔不易确定
Mars	基于 GPU 的 MapReduce 实现来执行数据和计算密集型任务	利用众多的 GPU 线程完成 Map 和 Reduce 的工作	易出现负载不均衡和发生写冲突

持其上的数据分析和机器学习,还用 Hadoop 构建了整个网站的数据仓库,进行日志分析和数据挖掘^[25]。淘宝是国内最先使用 Hadoop 的公司之一,其 Hadoop 系统用于存储并处理电子商务交易的相关数据^[26];百度使用 Hadoop 进行搜索日志分析和网页数据挖掘工作,百度在 Hadoop 上进行广泛应用并对它进行改进和调整,同时赞助了 HyperTable^[27] 的开发。

Hadoop 已取得非常突出的成绩,随着互联网的发展,新的业务模式还将不断涌现,其应用也会从互联网领域向电信、银行、电子商务、生物制药等领域拓展。

(2) Phoenix

Phoenix 是在共享内存系统上实现的 MapReduce,其应用主要包括:

- ①词频统计:统计文件集中每个词的共现频率;
- ②反向链接:创建 HTML 文件反向链接指数;
- ③矩阵乘法(Matrix Multiply):密集型整数矩阵乘法;
- ④字符串匹配:通过键值搜索文档;
- ⑤K-means:通过 K 均值迭代聚类算法,将三维数据点分成组;
- ⑥PCA (Principal Components Analysis):主成分分析矩阵;
- ⑦直方图:在一组图像中判定每个 RGB 分量的频率;
- ⑧线性回归 (Linear Regression):计算一组点的最佳拟合线^[6]。

(3) Disco

作为一个分布式处理的轻量级 MapReduce 实现框架,Disco 的用途主要是:解析、格式化、日志分析、聚类、概率建模、数据挖掘、全文索引和机器学习。目前 Disco 正被用于多种行业,解决具有挑战性的问题,涉及到大规模数据。Disco 商业级的应用如:在 Allston Trading,Disco 用于各种各样的历史研究和现代金融领域的实时举措;在 Chango,Disco 是分析和竞标广告市场的核心组件;在 Nokia,最大的数据分析集群运行的就是 Disco,对 Nokia 庞大的移动数据资产进行日常分析;在 Zemanta,Disco 用来处理关于维基百科和维基共享资源图像的上下文数据^[28]。

(4) Mars

Mars 是一个 GPU 上的 MapReduce 框架,包含了不同种类的 Web 数据分析如 Web 文档搜索(字符串匹配和倒排索引),Web 文档处理(K 均值、相似性评估和矩阵乘法)和 Web 日志分析(词频统计、网页访问计数和网页访问排名)等应用。由于 Mars 的编程接口是为图形处理而专门设计的,底层硬件比较复杂,造成用户编程难度较大,不利于其推广使用^[20]。

表4 主要实现平台比较

平台名称	难易程度	普及程度	应用领域
Hadoop	简单	较高	互联网领域、科研院所
Phoenix	简单	不高	企业计算、科学计算、人工智能、图像处理
Disco	简单	不高	商业领域、数据分析
Mars	复杂	较低	图形处理、Web 数据分析

表4 显示了 MapReduce 不同实现平台,在使用难易程度、普及程度及应用领域方面的比较。这4种实现平台,除 Hadoop 外,其他实现都没有得到广泛应用。

5 结 语

现实世界很多实例都可用 MapReduce 编程模型来表示,MapReduce 作为一个通用可扩展的并行处理模型,可有效地处理海量数据,不断地从中分析挖掘出有价值的信息。MapReduce 封装了并行处理、负载均衡、容错、数据本地化等技术难点的细节,对于没有并行或者分布式系统开发经验的程序员而言,MapReduce 库也易于使用。MapReduce 编程模型已成功应用于多个领域,但是,当前有关 MapReduce 的研究主要集中在其应用上,对于算法及算法效率提高和优化等方面的研究较少,还有待于重视,以使 MapReduce 能更好地被应用。

参考文献:

- [1] Dean J, Ghemawat S. MapReduce: Simplified Data Processing on Large Clusters[J]. *Communications of the ACM*, 2008, 51(1): 107 – 113.
- [2] White T. Hadoop: The Definitive Guide[M]. O' Reilly Media, 2009.
- [3] Ghemawat S, Gobioff H, Leung S. The Google File System[C]. In: *Proceedings of the 19th ACM SIGOPS Symposium on Operating Systems Principles (SOSP' 03)*, Bolton Landing, NY. New York, USA: ACM, 2003: 29 – 43.
- [4] MapReduce Tutorial [EB/OL]. [2011 – 08 – 19]. http://hadoop.apache.org/common/docs/current/mapred_tutorial.html.
- [5] EE382a: Advanced Processor Architecture[EB/OL]. [2011 – 08 – 20]. <https://courseware.stanford.edu/pg/courses/95981>.
- [6] Ranger C, Raghuraman R, Penmetsa A, et al. Evaluating MapReduce for Multi – core and Multiprocessor Systems[C]. In: *Proceedings of the 2007 IEEE 13th International Symposium on High Performance Computer Architecture (HPCA' 07)*. Washington, DC, USA: IEEE Computer Society, 2007: 13 – 24.
- [7] Technical Overview Disco Architecture [EB/OL]. [2011 – 12 – 22]. <http://discoproject.org/doc/overview.html>.
- [8] He B S, Fang W B, Luo Q, et al. Mars: A MapReduce Framework on Graphics Processors[C]. In: *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT' 08)*. New York, NY, USA: ACM, 2008: 260 – 269.
- [9] Mars: A MapReduce Framework on Graphics Processors [EB/OL]. [2011 – 08 – 20]. <http://www.cse.ust.hk/gpuqp/Mars.html>.
- [10] Hadoop Streaming [EB/OL]. [2011 – 12 – 23]. <http://hadoop.apache.org/common/docs/r0.15.2/streaming.html>.
- [11] Package org.apache.hadoop.mapred.pipes[EB/OL]. [2011 – 12 – 23]. <http://hadoop.apache.org/common/docs/current/api/org/apache/hadoop/mapred/pipes/package-summary.html>.
- [12] Leo S, Zanetti G. Pydoop: A Python MapReduce and HDFS API for Hadoop[C]. In: *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC' 10)*. New York, NY, USA: ACM, 2010: 819 – 825.
- [13] Pydoop [EB/OL]. [2011 – 12 – 26]. <http://sourceforge.net/projects/pydoop/>.
- [14] Style Guide for Disco Code [EB/OL]. [2011 – 12 – 26]. <http://discoproject.org/doc/howto/style.html>.
- [15] Programming Rules and Conventions [EB/OL]. [2011 – 12 – 26]. http://www.erlang.se/doc/programming_rules.shtml.
- [16] Style Guide for Python Code[EB/OL]. [2011 – 12 – 26]. <http://www.python.org/dev/peps/pep-0008>.
- [17] Lam C. Hadoop in Action[M]. Shelter Island, NY: Manning Publications Co., 2010.
- [18] POSIX Threads Programming [EB/OL]. [2011 – 12 – 29]. <https://computing.llnl.gov/tutorials/pthreads/>.
- [19] Disco Distributed File System[EB/OL]. [2011 – 12 – 29]. <http://discoproject.org/doc/howto/ddfs.html>.
- [20] 李建江, 崔健, 王聘, 等. MapReduce 并行编程模型研究综述[J]. *电子学报*, 2011, 39(11): 2635 – 2642. (Li Jianjiang, Cui Jian, Wang Dan, et al. Survey of MapReduce Parallel Programming Model[J]. *Chinese Journal of Electronics*, 2011, 39(11): 2635 – 2642.)
- [21] Langendoen K, Romein J, Bhoedjang R, et al. Integrating Polling, Interrupts, and Thread Management[C]. In: *Proceedings of the 6th Symposium on the Frontiers of Massively Parallel Computation*. Los Alamitos: IEEE Computer Society, 1996: 13 – 22.
- [22] Hadoop at Yahoo! [EB/OL]. [2012 – 01 – 07]. <http://developer.yahoo.com/hadoop/>.
- [23] PoweredBy – Hadoop Wiki[EB/OL]. [2012 – 01 – 07]. <http://wiki.apache.org/hadoop/PoweredBy>.
- [24] Hadoop at Last.fm [EB/OL]. [2012 – 01 – 08]. <http://www.slideshare.net/klbostee/hadoop-at-Lastfm>.
- [25] Facebook on Hadoop, Hive, HBase, and A/B Testing [EB/OL]. [2012 – 01 – 08]. <http://www.infoq.com/news/2010/07/facebook-hadoop-summit>.
- [26] Hadoop Archive – 淘宝共享数据平台 TBData. org [EB/OL]. [2012 – 02 – 08]. <http://www.tbdata.org/archives/category/cloud-computing/hadoop>. (Hadoop Archive; Taobao Shared Data Platform TBData. org [EB/OL]. [2012 – 01 – 08]. <http://www.tbdata.org/archives/category/cloud-computing/hadoop>.)
- [27] Hypertable: An Open Source, High Performance, Scalable Database[EB/OL]. [2011 – 08 – 19]. <http://hypertable.org/>.
- [28] Disco: Massive Data – minimal Code [EB/OL]. [2011 – 12 – 22]. <http://discoproject.org/about>.

(作者 E – mail: kangliyun945@163.com)